
Embedded Systems International

Team Embedded Code Review Worksheet (PD4)

Team Identifier: **G6**

Team Name (optional): **Nyx**

Date: **05/03/25**

Team Members Participating in the Code Review:

Kaylee Johnson, Olivia Blais, Lavanya Vangapandu, Rishita Kollu

Submit your document as a PDF file in Canvas under the corresponding project assignment.

Instructions for this assignment:

There are two code review checklists provided (files #3 and #4). As a team, select one of the checklist files for this assignment.

A checklist includes lists of rules that embedded code should adhere to. A particular checklist groups the rules into subsets of rules. A subset of rules is sometimes referred to as a perspective. For this assignment, the subsets are shown in the checklist in each file (file #3 has subsets labeled Reviewer #, and file #4 has subsets labeled Function, Style, etc.).

Each student on a team should select one of the subsets in the team's checklist (e.g., a student might select the Reviewer #1 subset or the Style subset). Then a student picks 2-3 rules listed in their subset. This keeps the workload reasonable for this mock review. Students on the same team may select the same subset if there are more students than subsets in a checklist. However, it would be useful to pick different rules.

Each student keeps their chosen rules in mind when reviewing the team's code (whatever lines of code were selected). A student should write down any issues found in the code in relation to the rules they are responsible for. You don't need to improve the code. You are simply reviewing code with particular rules in mind and identifying potential issues.

Note: If you are unsure about what a rule means, write down what you think it means and apply the rule as you defined it.

Select one of these checklists for the code review:

___ Peer Review Checklist: Embedded C Code (P. Koopman, ECE CMU, 2018)

X Embedded System Code Review Checklist (G. Khattak & P. Koopman, July 2012)

These checklists are items 3 and 4 in the Code Review resources for this assignment:

- 1-Better Embedded System SW-Embedded System Code Review Checklist.pdf
- 2-PeerReview-selected-slides-CMU.pdf
- 3-peer_review_checklist-CMU.pdf
- 4-code_review_checklist_v1_010.pdf

1. Select 30-40 lines of code developed for your project.

```
double move_forward(oi_t *sensor_data, double distance_mm) {  
    double distance_traveled = 0;  
    oi_setWheels(FORWARD_SPEED, FORWARD_SPEED);  
    while (distance_traveled < distance_mm) {  
        oi_update(sensor_data);  
        distance_traveled += sensor_data->distance;  
        lcd_printf("D:%.0f\nBL:%d BR:%d", distance_traveled,  
sensor_data->bumpLeft, sensor_data->bumpRight);  
        if (sensor_data->bumpLeft || sensor_data->bumpRight) {  
            stop_robot();  
            move_backward(sensor_data, 50);  
            uart_sendStr("BUMP DETECTED\n");  
            alertBump();  
            return distance_traveled - 50;  
        }  
    }  
}
```

```

        if (cliff_front_right_on_tape(sensor_data) ||
            cliff_right_on_tape(sensor_data) ||
                cliff_front_left_on_tape(sensor_data) ||
            cliff_left_on_tape(sensor_data)) {

            stop_robot();

            uart_sendStr("TAPE DETECTED\n");

            alertTape();

            return distance_traveled;

        }

        if (sensor_data-> cliffRight || sensor_data-> cliffFrontRight
            || sensor_data-> cliffFrontLeft || sensor_data-> cliffLeft){

            stop_robot();

            uart_sendStr("HOLE DETECTED\n");

            alertHole();

            return distance_traveled;

        }

    }

    stop_robot();

    return distance_traveled;

}

```

2. From your selected checklist, identify *perspectives* to be taken by team members. Refer to items 1 and 2 in the Code Review resources to learn about perspectives.

Lavanya - Function:

- F-1. Does the code match the design and the system requirements?
- F-2. Does the code do what it should be doing?
- F-3. Does the code do anything it should not be doing?
- F-4. Can the code be made simpler while still doing what it needs to do?
- F-5. Are available building blocks used when appropriate? (algorithms, data structures, types, templates, libraries, RTOS functions)
- F-6. Does the code use good patterns and abstractions? (e.g., state charts, no copy-and paste)

- F-7. Can this function be written with a single point of exit? (no returns in middle of function)
- F-8. Are all variables initialized before use?
- F-9. Are there unused variables?
- F-10. Is each function doing only one thing? (Does it make sense to break it down into smaller modules that each do something different?)

Kaylee - Style:

- S-1. Does the code follow the style guide for this project?
- S-2. Is the header information for each file and each function descriptive enough?
- S-3. Is there an appropriate amount of comments? (frequency, location, and level of detail)
- S-4. Is the code well structured? (typographically and functionally)
- S-5. Are the variable and function names descriptive and consistent in style?
- S-6. Are "magic numbers" avoided? (use named constants rather than numbers)
- S-7. Is there any "dead code" (commented out code or unreachable code) that should be removed?
- S-8. Is it possible to remove any of the assembly language code, if present?
- S-9. Is the code too tricky? (Did you have to think hard to understand what it does?)
- S-10. Did you have to ask the author what the code does? (code should be self-explanatory)

Rishita - Robustness & Safety:

- E-1. Are input parameters checked for proper values (sanity checking)?
- E-2. Are error return codes/exceptions generated and passed back up to the calling function?
- E-3. Are error return codes/exceptions handled by the calling function?
- E-4. Are null pointers and negative numbers handled properly?
- E-5. Do switch statements have a default clause used for error detection?
- E-6. Are arrays checked for out of range indexing? Are pointers similarly checked?
- E-7. Is garbage collection being done properly, especially for errors/exceptions?
- E-8. Is there a chance of mathematical overflow/underflow?
- E-9. Are error conditions checked and logged? Are the error messages/codes meaningful?
- E-10. Would an error handling structure such as try/catch be useful? (depends upon language)

Olivia - ARCHITECTURE

- A-1. Is the function too long? (e.g., longer than fits on one printed page)
- A-2. Can this code be reused? Should it be reusing something else?
- A-3. Is there minimal use of global variables? Do all variables have minimum scope?
- A-4. Are classes and functions that are doing related things grouped appropriately? (cohesion)
- A-5. Is the code portable? (especially variable sizes, e.g., "int32" instead of "long")
- A-6. Are specific types used when possible? (e.g., "unsigned" and typedef, not just "int")
- A-7. Are there any if/else structures nested more than two deep? (consecutive "else if" is OK)
- A-8. Are there nested switch or case statements? (they should never be nested)

3. From your selected checklist, identify several specific rules to be applied during the code review. Each team member should **choose two or three rules** and focus on those given the limited time available for a short mock review session.

Kaylee – Style Issues:

- **S-2. Is the header information for each file and each function descriptive enough?**
 - o There is no direct header for this specific function, and the header that would be applied to all movement logic is very vague, leaving it up to the programmer to read through every line of code individually.
- **S-3. Is there an appropriate amount of comments? (frequency, location, and level of detail)**
 - o There are no comments in this section of code; in other sections that were not selected, there are minimal comments explaining basic logic behind sections of code.
- **S-4. Is the code well structured? (typographically and functionally)**
 - o Despite the lack of comments and headers, the code is formatted in an easy-to-read structure and allows the programmer to see clearly which parts function together.

Olivia Blais:

- **A-2. Can this code be reused? Should it be reusing something else?**
 - o This snippet of code does well to reuse code from something else to minimize the code used for its functionality. We use `move_backward()` and `stop_robot()`, which help reduce the sensor logic. We could create a separate function for the emergency handling for tape, hole and bump to simplify the function and create an opportunity to reuse the emergency logic somewhere else.
- **A-6. Are specific types used when possible? (e.g., "unsigned" and typedef, not just "int")**
 - o After reviewing the code, I noticed that we are using a double for our movement distance when we could just use an int instead. In other parts of our code, when we call the function, we normally use the int values.
- **A-7. Are there any if/else structures nested more than two deep? (consecutive "else if" is OK)**
 - o We properly use nested if/else statements by not nesting them more than two deep, keeping the code readable and well structured.

Rishita - Robustness & Safety:

- **E-1. Are input parameters checked for proper values (sanity checking)?**
 - o The code does not check if `sensor_data` is valid before using it. If `sensor_data` is null, then the function could crash when it tries to call `oi_update(sensor_data)` or read sensor values. Also, `distance_mm` is not checked, so if a negative distance is passed in, the function will not really move but it still returns a value.
- **E-4. Are null pointers and negative numbers handled properly?**

- Null pointers are not handled in this function. Since `sensor_data` is a pointer, it should be checked before it is used. Negative numbers are also not handled clearly. Since this function is called `move_forward`, a negative distance should probably be rejected or handled before starting the movement.
- **E-9. Are error conditions checked and logged? Are the error messages/codes meaningful?**
 - The function does log some error/safety conditions through UART, like "BUMP DETECTED", "TAPE DETECTED", and "HOLE DETECTED". These messages are meaningful and help tell what stopped the robot. However, the function does not log invalid input errors, like if `sensor_data` is null or if the distance is invalid.

Lavanya – Function:

- **F-2. Does the code do what it should be doing?**
 - The code does do what it should be doing. The function is meant to focus on the bot's movement forward with the given distance and handle any interruptions during its movement. The function should return the distance the bot moved, accounting for any obstacles it ran into along the way.
- **F-7. Can this function be written with a single point of exit? (no returns in middle of function)**
 - Currently, the function is written with four points of exit. We set this function up to break when unforeseen obstacles interrupt the robot's movement. We did this to improve the execution speed since we know that if any interruptions occur, the rest of the function is not needed at that moment.
- **F-10. Is each function doing only one thing? (Does it make sense to break it down into smaller modules that each do something different?)**
 - In retrospect, we could have broken the function into smaller modules. Some potential modules would be to move the bump, hole, and tape detection into their own functions. This would simplify the logic and make the function more straightforward. Breaking it into separate functions would allow us to reuse the detection logic in other parts of the codebase.

4. Record any issues found related to the specific rules considered.

In the style's subset, some issues we notice are the names of other functions used inside the movement function (`alertHole()`, `alertBump()`, `alertTape()`). These do not follow the naming conventions set, nor are they used throughout most of our code. The function names should normally use the `snake_case` style with the underscores separating the words. Most of our function names are descriptive and short, ensuring easy code readability. In this snippet of code, we are lacking comments, which are important for understanding code. This would help teammates and anyone else reading the code to quickly understand the intention of the logic currently written, and it can help with debugging later on.

For rules in the Architecture subset, we found a couple of issues. One issue, as mentioned before, is that we could improve the reusability of the code and be more intentional with the types of variables used. We initially had the logic inside the `stop_robot()` function inside the movement function, causing redundant code.

Moving on to issues with the function subset, we initially had the bot just moving the distance given without any obstacle handling. This was not a realistic use of the function, as the field and environment will have obstacles that the bot will need to account for in the moment. Once we realized this, we initially handled bump detection and later implemented hole and tape detection that utilizes the cliff sensors on the bot.

In addition, some issues related to robustness and safety occurred at the beginning of our development, involving the cliff sensors and the distance units. For the cliff sensors, we had to figure out how to use the oi interface's cliff and cliff signal readings. We figured out that the reading from the signal gave us the hole detection and the cliff readings of the raw IR values. We created thresholds for the tape detection using the raw IR values. Additionally, we initially assumed the distance was measured in centimeters, but it was actually measured in millimeters. Our accuracy with the movement function improved after fixing this issue. In order to test these values, we added an LCD print statement for real-time evaluation of these values.